Information_security_ITMO

Выполнил: Батаргин Егор Александрович

Задание

Дополнительные потоки данных в файлах NTFS.

- На томе NTFS создать файл (например, "test.txt")
- Выполнить команду "dir > test.txt:aaa.txt" (при этом вывод будет перенаправлен в поток aaa.txt файла test.txt; все программы работают только с основным потоком файла)
- вывести содержание созданного потока на экран "more < test.txt:aaa.txt" открыть в notepad "notepad test.txt:aaa.txt"

Задание: 1.0. Сравнить размер файла до и после создания новых потоков. 1.1. Сравнить свободное место на логич. диске до и после (лучше использовать достаточно большие файлы) 1.2. Выводя в поток данные (известного размера) и сравнивая остаток свободного места на лог. диске (использовать отличный от системного диск) построить зависимость свободного места от размера записанных данных. (в байтах) 2.0. Вывести содержание потока test.txt:aaa.txt в другой файл (testdir.txt) 3.0. Посмотреть как работают другие комнды cmd.exe/command.com с потоками (например: type, echo, etc.) 4.0. Посмотреть как работают другие программы с потоками (notepad/hiew/word/etc.)

```
Дополнительно (необязательное):
```

1. Предложить способ хранения и извлечения бинарных (исполняемых) файлов в дополнительных потоках файлов. При этом подразумевается использования стандартных средств (входящих в дистриб. ОС Windows 2000/XP)
2. Найти дополнительную информацию по потокам (и по использованию их на программном уровне, win32/native API/etc.)
3. Выяснить совпадают ли разграничения доступа к файлу с разграничениями доступа к потоку.
4. Написать программу (на win32 использующую потоки для хранения данных [чтение - запись])

Этап 1

Создадим пустой файл `test.txt`. Согласно теории, данные не должны будут записаться в этот файл. На пустом файле как раз это будет видно. Главное, выполнять не в `PowerShell`, а в `cmd`

```
type nul > test.txt
```

Записываем данные в поток.

```
dir > test.txt:aaa.txt
```

И проверим

```
more < test.txt:aaa.txt
```

Вот такой вывод:

```

Том в устройстве C не имеет метки. Серийный номер тома: F2AF-0EDB

Содержимое папки C:\Users\batar\Yandex.Disk\Университет ИТМО\4 курс\ИБ

12.10.2026 18:25

    . 12.10.2026 18:03
        .. 12.10.2026 18:27 0 test.txt 12.10.2026 17:57 2 764 Лабораторная работа 1 (задание).TXT
        2 файлов 2 764 байт 2 папок 4 193 083 392 байт свободно

        Что по выводу видно:
        - Файл `test.txt` не изменился в размере
        - Данные в потоке есть


        Итоге по 1.0:
        Размер файла (его основного потока) не изменился — остался 0 байт. При этом данные
        реально записались и хранятся в NTFS в скрытом потоке. Они занимают место на диске,
        хотя файл выглядит пустым.

        ## Этап 1.1


        Узнаем данные по свободному месту до (запускаем от имени админа):
```

fsutil volume diskfree C:

```
У меня вывод такой
```

Всего свободно (байт) : 1 228 361 728 (1,1 ГБ) Всего (байт) : 383 476 822 016 (357,1 ГБ) Всего неиспользованной квоты (байт) : 1 228 361 728 (1,1 ГБ) Недоступно в пуле (байт) : 0 (0,0 КБ) Использованная квота пула (байт) : 0 (0,0 КБ) Использовано (байт) : 376 669 990 912 (350,8 ГБ) Всего зарезервировано (байт) : 5 578 469 376 (5,2 ГБ) Зарезервировано для хранилища для тома (байт) : 5 386 977 280 (5,0 ГБ) Выделено и доступно (байт) : 0 (0,0 КБ) Доступно в пуле (байт) : 0 (0,0 КБ)

```
Создадим файл в 50 МБ
```

fsutil file createnew big.bin 52428800

```
Запишем данные в поток
```

type big.bin > test.txt:aaa.txt

```
Повторим команду для просмотра сведений по пространству
```

Всего свободно (байт) : 1 121 509 376 (1,0 ГБ) Всего (байт) : 383 476 822 016 (357,1 ГБ) Всего неиспользованной квоты (байт) : 1 121 509 376 (1,0 ГБ) Недоступно в пуле (байт) : 0 (0,0 КБ) Использованная квота пула (байт) : 0 (0,0 КБ) Использовано (байт) : 376 775 659 520 (350,9 ГБ) Всего зарезервировано (байт) : 5 579 653 120 (5,2 ГБ) Зарезервировано для хранилища для тома (байт) : 5 386 977 280 (5,0 ГБ) Выделено и доступно (байт) : 0 (0,0 КБ) Доступно в пуле (байт) : 0 (0,0 КБ)

```
По итогу
```

Убыль = ДО − ПОСЛЕ Убыль = 1 228 361 728 − 1 121 509 376 = 106 852 352 байт

```
Размер test.txt
```

12.10.2026 19:32 0 test.txt 1 файлов 0 байт 0 папок 1 111 773 184 байт свободно

```
Вывод такой: место уменьшилось, но сам файл остался равен 0

## Этап 1.2
Для построения зависимости использовалась флешка (диск E:), отформатированная в NTFS.

Тест 1 (5 МБ). Создаем пустой файл, замеряем место, создаем файл 5 МБ, пишем в поток,
удаляем исходник и снова замеряем.
```

type nul > test.txt fsutil volume diskfree E: fsutil file createnew big.bin 5242880 type big.bin > test.txt:aaa.txt del big.bin fsutil volume diskfree E:

```
Результат:
```

Всего свободно (байт) : 34 285 015 040

```
Всего свободно (байт) : 34 279 772 160
Убыль: 34 285 015 040 – 34 279 772 160 = 5 242 880 байт.

Тест 2 (10 МБ).
```

type nul > test.txt fsutil volume diskfree E: fsutil file createnew big.bin 10485760 type big.bin > test.txt:aaa.txt del big.bin fsutil volume diskfree E:

```
Результат:
```

Всего свободно (байт) : 34 285 015 040

```
Всего свободно (байт) : 34 274 529 280
Убыль: 34 285 015 040 – 34 274 529 280 = 10 485 760 байт.

Тест 3 (20 МБ).
```

type nul > test.txt fsutil volume diskfree E: fsutil file createnew big.bin 20971520 type big.bin > test.txt:aaa.txt del big.bin fsutil volume diskfree E:

```
Результат:
```

Всего свободно (байт) : 34 285 015 040

```
Всего свободно (байт) : 34 264 043 520
Убыль: 34 285 015 040 – 34 264 043 520 = 20 971 520 байт.

Вывод: Убыль свободного места на диске в точности равна размеру данных, записанных в
поток. Зависимость прямо пропорциональна. При этом размер файла test.txt остается
равен 0 байт.

## Этап 2.0

Выполняем вывод содержания потока в другой файл с помощью PowerShell (так как cmd не
справляется с большими бинарными данными).
```

Get-Content -Path test.txt -Stream aaa.txt -Encoding Byte -ReadCount 0 | Set-Content -Path testdir.txt -Encoding Byte Проверяем размер полученного файла.

```
```

dir testdir.txt

```
Результат:
```

26.10.2026 15:30 52 428 800 testdir.txt

```
Вывод: Содержимое скрытого потока успешно извлекается в обычный файл, размер
извлеченного файла (52 428 800 байт) в точности совпадает с размером исходных данных,
записанных в поток.

## Этап 3.0

Проверяем работу стандартных команд cmd.exe с альтернативными потоками. Создаем
текстовый поток с помощью команды `echo`:
 ```
echo "Секретные данные" > test.txt:secret.txt
```

Пробуем прочитать поток командой `type` через оператор перенаправления ввода:

```
type < test.txt:secret.txt
```

Результат:

```
Ошибка в синтаксисе команды.
```

Причина в том, что команда `type` в `cmd.exe` изначально не поддерживает синтаксис альтернативных потоков данных (ADS) вида `файл:поток`

Пробуем прочитать поток командой `more` через оператор перенаправления ввода:

```
more < test.txt:secret.txt
```

Результат:

```
"Секретные данные"
```

Проверяем, отображает ли команда `dir` наличие скрытого потока:

```
dir test.txt
```

Результат:

```
26.10.2026 16:47 0 test.txt
```

Проверяем работу оператора дозаписи `>>` с помощью `echo` и читаем результат через `more`:

```
echo "Вторая строка" >> test.txt:secret.txt
more < test.txt:secret.txt
```

Результат:

```
"Секретные данные"
"Вторая строка"
```
```

Вывод: Стандартная команда `echo` поддерживает запись и дозапись в альтернативные потоки с помощью операторов `>` и `>>`. Команда `type` не поддерживает работу с ADS ни напрямую, ни через `<`. Команда `more` корректно извлекает данные из потока при использовании перенаправления ввода (`more < файл:поток`). Команда `dir` не отображает наличие альтернативных потоков, показывая только нулевой размер основного файла.

Этап 4.0

Проверяем работу графических программ с альтернативными потоками данных. Открываем папку с файлом `test.txt` в Проводнике Windows. *Результат:* Проводник отображает файл `test.txt` размером **0 КБ**. Скрытые потоки в графическом интерфейсе не видны.

Для записи читаемого русского текста в поток использовалась команда PowerShell с параметром `-Stream` (так как синтаксис `файл:поток` в PowerShell не работает, он воспринимает двоеточие как имя диска):

```
powershell -Command "Set-Content -Path test.txt -Stream secret.txt -Value 'Секретные данные из потока' -Encoding Unicode"
```

Запускаем Блокнот (Notepad) из командной строки, передавая ему в качестве аргумента имя скрытого потока: notepad test.txt:secret.txt

Результат: Секретные данные из потока

Вывод: Проводник Windows полностью игнорирует альтернативные потоки и показывает только основной файл нулевого размера. Однако Блокнот способен корректно открыть и отобразить содержимое потока, если запустить его с синтаксисом `файл:поток`. При этом кодировка имеет значение: для корректного отображения русского текста в Блокноте поток был записан в кодировке Unicode с помощью `Set-Content` с параметром `-Stream`. Это подтверждает, что ADS являются полноценным хранилищем данных, доступным не только для командной строки, но и для некоторых GUI-приложений.

Дополнительный пункт 1. Хранение и извлечение бинарных файлов

Для демонстрации скрытого хранения исполняемых файлов в альтернативных потоках был использован системный калькулятор Windows (`C:\Windows\System32\calc.exe`).

Создаем файл-приманку `readme.txt` и перенаправляем в его скрытый поток бинарный файл:

```
type nul > readme.txt
type C:\Windows\System32\calc.exe > readme.txt:calc.exe
```

Проверяем размер файла-приманки:

```
dir readme.txt
```

Результат:

```
26.10.2026 17:02 0 readme.txt
```

Файл снаружи кажется абсолютно пустым (0 байт).

Извлекаем скрытый бинарный файл обратно с помощью PowerShell (для корректной работы с бинарными данными):

```
powershell -Command "Get-Content -Path readme.txt -Stream calc.exe -Encoding Byte -
ReadCount 0 | Set-Content -Path extracted_calc.exe -Encoding Byte"
```

Проверяем размер извлеченного файла:

```
dir extracted_calc.exe
```

Результат:

```
26.10.2026 17:02 49 152 extracted_calc.exe
```

Дополнительный пункт 3. Разграничение доступа к файлу и потоку

Для выяснения того, совпадают ли разграничения доступа к файлу с разграничениями доступа к потоку, был проанализирован механизм работы списков управления доступом (ACL) в файловой системе NTFS.

Суть механизма: В Windows права доступа (ACL) назначаются на **дескриптор безопасности самого файла**, а не на отдельные его атрибуты или потоки. В системе безопасности Windows не существует отдельных разрешений вида «Чтение потока» или «Запись потока». Все права (на чтение, запись, изменение, удаление) применяются к файлу как к единому объекту.

Практический вывод: Разграничения доступа к файлу и к его скрытым потокам **полностью совпадают**. Если пользователь имеет право на чтение файла `test.txt`, он автоматически получает право на чтение всех его альтернативных потоков (например, `test.txt:secret.txt` или `test.txt:aaa.txt`).

Невозможно настроить политику безопасности так, чтобы, например, основной поток файла был доступен для чтения всем пользователям, а скрытый поток — только администратору. Из-за этого механизм ADS нельзя использовать как средство дополнительного разграничения доступа (например, для сокрытия конфиденциальных данных от коллег, у которых есть доступ к общей папке).

Однако с точки зрения уязвимости, это означает обратное: антивирусы и средства защиты, работающие под учетной записью пользователя, имеют техническую возможность прочитать скрытые потоки (если умеют), но вредоносный код может маскироваться, рассчитывая на то, что легитимные программы проверяют только

основной поток. Файл извлечен без потерь (размер 49 152 байта) и успешно запускается командой `start extracted_calc.exe`.

Вывод: Альтернативные потоки NTFS позволяют скрыто хранить и извлекать бинарные (исполняемые) файлы любого размера. Сторонние программы и Проводник Windows не видят наличия посторонних данных внутри пустого текстового файла. Это создает серьезную угрозу информационной безопасности, так как вредоносное ПО (вирусы, трояны, скрипты) может быть скрыто в безобидных файлах (картинках, текстах) и незаметно для антивирусов, которые проверяют только основной поток, запущено на системе.

Общий вывод по лабораторной работе

В ходе выполнения лабораторной работы были изучены механизмы работы альтернативных потоков данных (Alternate Data Streams, ADS) в файловой системе NTFS.

Основные результаты: 1. Экспериментально подтверждено, что при создании и записи данных в скрытый поток, размер самого файла (основного потока) не изменяется и отображается как 0 байт. Однако данные в потоке реально существуют и занимают дисковое пространство. 2. Исследование зависимости свободного места показало, что альтернативные потоки занимают место на диске строго пропорционально объёму записанных в них данных (коэффициент 1:1), при этом накладные расходы файловой системы минимальны. 3. Доказано, что содержимое потоков можно не только прочитать на экране (с помощью `more`), но и корректно извлечь обратно в обычный файл без потери данных. 4. Выявлено, что стандартные команды cmd.exe (например, `echo`, `type`, `dir`) и графические программы (Проводник Windows) не отображают наличие потоков. При этом некоторые программы (Блокнот) способны открыть поток, если передать ему путь в формате `файл:поток`. 5. Дополнительно продемонстрирована возможность скрытого хранения и успешного запуска бинарных исполняемых файлов (на примере `calc.exe`), спрятанных внутри безобидного пустого текстового файла.

Значение для информационной безопасности: Механизм ADS представляет собой серьезную угрозу безопасности. Злоумышленники могут использовать потоки для сокрытия вредоносного кода (вирусов, троянов, скриптов), конфиденциальных данных или логов от системных администраторов. Проводник и базовые сканеры не видят эти данные. Кроме того, было выяснено, что разграничения прав доступа не позволяют защитить поток отдельно от файла (права наследуются), что исключает использование ADS для легитимного разделения прав, но позволяет обходить некоторые механизмы защиты. Для противодействия данной угрозе необходимо использовать специализированный инструментарий (например, утилиту `streams.exe` от Sysinternals) для поиска и очистки скрытых потоков на дисках.